

最详细的DSHOT协议介绍



科G栈

2022年04月08日 19:43

收录于文集

穿越机 · 13篇

⋮



目录



53



12



103



1



39

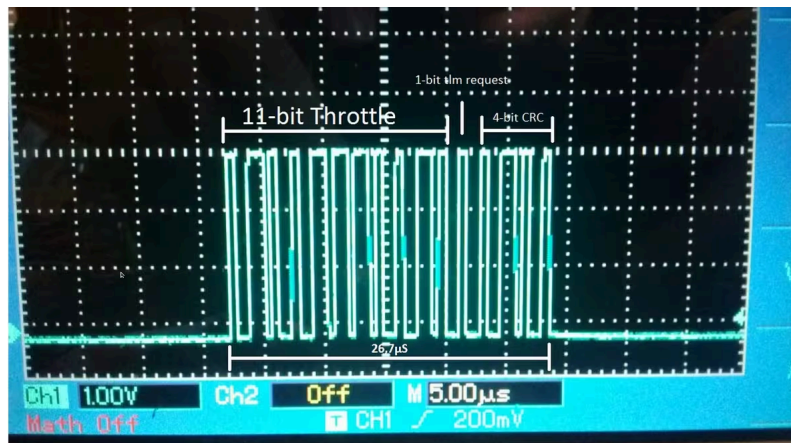
Dshot是啥？

DSHOT是一种数字ESC协议，特点是高可靠性、高扩展性和高性能。由Felix (KISS)设计开发，在与Boris 和 betaflight 团队的其他成员不断完善下，最终成为一种成熟稳定的协议。Steffen (BLHeli)将该协议移植到了BLHeli_S。目前已经成为穿越机的主流电调协议。

目前DSHOT协议有4种，DSHOT150，DSHOT300，DSHOT600，DSHOT1200。后面的数字表示传输速率，即每秒多少千比特位。对于DSHOT600，它每秒可以发送 600K比特。引入 DSHOT300 和 DSHOT150 是为了确保支持功能较弱的旧ESC，以便飞行者能够受益于数字精度，DSHOT1200是较新的协议，有些电调可能还不支持。

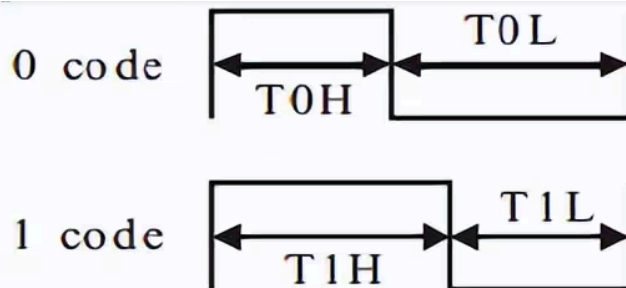
技术细节

下图是一帧DSHOT600的波形图，它由16个比特位构成，其中前11位是油门信号，第12位是telemetry请求标志（tlm需要电调硬件支持），后四位是CRC校验位。速度600kbits/s，一帧信号的长度为26.7us。



帧与帧之间需要一点间隔（2-3us），以区别不同的帧信号，理论最高刷新率为33khz，现实基本上达不到这么高的频率。对于betaflight最高是pid的循环频率8khz。

DSHOT的比特位如下图所示：



目录

53

12

103

1

39

对于DSHOT600，整个比特位的长度为1.67us（T0H+T0L或T1H+T1L），0的高电平时间为625ns，1的高电平时间是1250ns。如果你用过ws2812的RGB彩灯，你应该对这种信号比较熟悉。

11位油门值可以达到2048的分辨率，实际使用48–2047表示油门值，所以油门信号是2000的分辨率，0是上电后的默认值（锁定值），1–47表示一些命令和设置，一些值的意义如下：

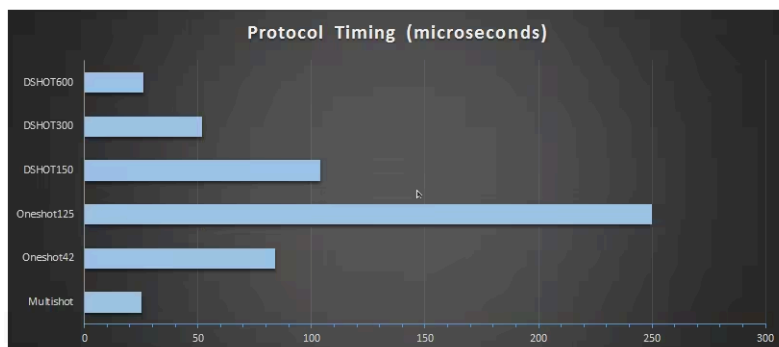
最后的校验位通过前12位的信号分成3组半字节异或得到。

相比模拟信号的优势

非数字PWM协议如Multishot、Oneshot42、Oneshot125等，都是依靠脉冲宽度来指示油门位置的，这种方式有许多明显的缺点，比如抖动，脉冲宽度越短，抖动的可能性和影响就越大；脉冲宽度的变化也会导致无法确定零油门和最大油门，所以在使用模拟信号时需要校准电调；模拟信号也没法发送控制命令，而且分辨率受系统条件限制也不会太高。

数字信号则是准确的，即便信号受到某些影响导致高低电平时长有些变化，但只要变化在可以接收的范围，0和1的识别就不成问题。如果信号损坏，ESC还可以通过校验检测到它，从而扔掉错误帧；DSHOT不需要校准，在数字世界中，零就是一个零；DSHOT可以达到2000的分辨率，这点所有的模拟信号都达不到；数字信号的功能也可以更加丰富，保留的1–47信号可以提供丰富实用的设置命令。

当然DSHOT并不是效率最高的协议，下图表示当前几种协议的全油门时间：



Oneshot42	42	42	84
Oneshot125	125	125	250
Standard	1000	1000	2000
DSHOT150	104*	104*	104
DSHOT300	52*	52*	52
DSHOT600	26*	26*	26

📖
目录

👍
53

🏷️
12

☆
103

🔖
1

💬
39

图中用时最短的是Multishot协议，但是DSHOT600一帧时间仅比 Multishot慢1微秒，DSHOT150也与Oneshot42差不多。不过如果DSHOT1200得到普及，那DSHOT的优势将无懈可击。DSHOT实际上没有命令最小和最大持续时间的真正概念，对于数字协议，0与2047所需的通信时间相同。

双向dshot

Joe Lucid更进一步引入了双向 DSHOT协议。



双向DSHOT信号使用反转电平（空闲为 1）。FC 到 ESC 使用 DSHOT 帧，但最低 4 位是其他半字节异或后的反码（正常DSHOT的校验是不反码的）。

（complement不确定翻译为反码还是补码，不过根据实际波形推断应该是反码，欢迎大家纠正）

ESC在收到DSHOT帧30us后发回telemetry帧。逻辑上telemetry帧是一个16位的值，而最低的 4 位依然是其他半字节异或后的反码。

高12位包含以下按位编码的 eperiod (1/erps):

mmmeeeeeeeee

前3位代表左移的次数，后9位表示基数。这样就可以表示周期范围1~65408us, (Which translates to a min e-frequency of 15.29 hz or for 14 pole motors 3.82 hz, 原文这个3.82hz不知道怎么算出来的)

之后这个16位的数按照下面的GCR编码转换成了一个20位的数。之所以要这么转一下，作者的解释是为了适配BLHeli_S电调。

0000	11001	1000	11010
0001	11011	1001	01001
0010	10010	1010	01010
0011	10011	1011	01011
0100	11101	1100	11110
0101	10101	1101	01101
0110	10110	1110	01110
0111	10111	1111	01111

📖
目录

👍
53

🏷️
12

☆
103

🔖
1

💬
39

这个20位的值特点是不超过两个连续零。之后再映射为一个21位的值，首先高位添加一个0，20位的值依次由高到低传入，如果传入值为1，则更改下一位的位值，否则重复之前的位值。例如：

1 0 1 1 1 0 0 1 1 0 将成为 0 1 1 0 1 0 0 0 1 0 0。

然后以DSHOT比特率*5/4的比特率不反转发送。因此，DSHOT300使用375 kbit，DSHOT600使用750 kbit。

Esc需要在40us + 一个DSHOT帧长度后准备好才能接收下一个DSHOT数据包。

双向DSHOT这里比较复杂，我理解的不一定正确，感兴趣的小伙伴可以查看官方文档。

参考文档：

<https://github.com/betaflight/betaflight/wiki/Dshot>

<https://github.com/betaflight/betaflight/pull/8554>

[https://en.wikipedia.org/wiki/Run-length_limited#GCR:\(0,2\)_RLL](https://en.wikipedia.org/wiki/Run-length_limited#GCR:(0,2)_RLL)

<https://github.com/betaflight/betaflight/wiki/DSHOT-ESC-Protocol>

附：stm32 dma 核心代码

```
#define ESC_BIT_0    11
#define ESC_BIT_1    22
#define ESC_CMD_BUF_LEN 18
uint16_t ESC_CMD[ESC_CMD_BUF_LEN]={0};

static uint16_t prepareDshotPacket(const uint16_t value, int8_t request)
{
    // 油门大小为11位 所以这里先左移一位 添加上请求回传标志共12位
    uint16_t packet = (value << 1) | (requestTelemetry ? 1 : 0)

    // 将12位数据分为3组 每组4位，进行异或
    // compute checksum
    int csum = 0;
    int csum_data = packet;
    for (int i = 0; i < 3; i++) {
        csum ^= csum_data; // xor data by nibbles
    }
}
```

```
csum &= 0xf;
// append checksum 将CRC添加到后四位
packet = (packet << 4) | csum;
return packet;
}

static void pwmWriteDigital(uint16_t *esc_cmd, uint16_t value)
{
    value = (value > 2047) ? 2047 : value;
    value = prepareDshotPacket(value, 0);
    esc_cmd[0] = (value & 0x8000) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[1] = (value & 0x4000) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[2] = (value & 0x2000) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[3] = (value & 0x1000) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[4] = (value & 0x0800) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[5] = (value & 0x0400) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[6] = (value & 0x0200) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[7] = (value & 0x0100) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[8] = (value & 0x0080) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[9] = (value & 0x0040) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[10] = (value & 0x0020) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[11] = (value & 0x0010) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[12] = (value & 0x8) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[13] = (value & 0x4) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[14] = (value & 0x2) ? ESC_BIT_1 : ESC_BIT_0;
    esc_cmd[15] = (value & 0x1) ? ESC_BIT_1 : ESC_BIT_0;

    HAL_TIM_PWM_Start_DMA(&htim1, TIM_CHANNEL_1, (uint32_t *)esc_cm
}
```



目录



53



12



103



1



39

穿越机 fpv 电调 dshot

分享至



投诉或建议

评论 39 赞与转发 54

最热 | 最新



进来和UP唠会嗑呗



Ncerzzk LV5

请教一下，双向Dshot的电路上是怎么连接的呢？需要设置成OD然后接上拉电阻吗？如果是两边直接推挽相连，在通信的过程中，如果出了差错，一个输出高电平另一边输出低电平岂不是短路了？

2022-06-17 00:11 回复

共 3 条回复，点击查看



码前小卒 LV5

目前网上的资料对于betaflight都是不太对的上，大多数人认为是用PWM+DMA的方式去产生DSHOT其实betaflight是用GPIO+DMA的方式。

2022-04-18 14:43 4 回复

共 6 条回复，点击查看

凌启科技-1 LV5

你好，驱动dshot600电调的时候需要发送命令吗，我的直接发送解析后的pwm值电机没有驱动起来

zhanyh89 LV4

请问双向DSHOT的0油门是多少，我用单片机发到双向DSHOT信号，开启正弦启动后发48电机就会启

2024-12-31 09:58 1 0 回复

共 1 条回复，点击查看

xy3dg12 LV5

telemetry里16位转20位，我理解是类似4B5B编码，防止过多连续的0或1出现在通讯线路中

2023-11-09 11:18 1 0 回复

共 1 条回复，点击查看

CuiRiooo LV5

up主你好，dshot设置电机转向有试过吗。无论是发送7，8，20，21都没办法改变电机的方向

2024-11-06 19:43 1 0 回复

北万油皮主 LV5

这么巧 碰到我写的代码。。。

2023-11-24 16:29 3 0 回复

随风FPV LV6

感谢分享，双向DSHOT具体实现细节太难没看懂，理解了大概原理和作用——在有限的通讯信息载体中，巧妙的信息“装载方法”增加了单位通讯量，使得电调和电机的通讯更高效。就像一辆卡车车厢，大小固定，设计好装卸流程就可以装载得更多，装卸更快。如果理解的不对，欢迎指正🐶

2022-04-12 09:43 1 0 回复

琴声悠扬荡我心 LV5

前3位代表左移的次数，后9位表示基数。这样就可以表示周期范围1~65408us.....这个怎么算出来的

2024-12-18 02:41 1 0 回复

musso垚 LV4

太详细了 只是没有真看懂🤔

2022-04-08 22:07 1 0 回复

共 1 条回复，点击查看

异史氏-Official LV5

请问一下对于四合一电调又是如何实现独立控制每个电机的呢？

2022-04-13 15:27 1 0 回复

共 1 条回复，点击查看

优雅dr老白 LV5

双向Dshot有什么参考文档吗

2022-10-25 16:46 1 0 回复

共 1 条回复，点击查看

pluto978 LV5

不懂，学学。

2022-04-10 14:30 1 0 回复

📖 目录

👍 53

🔒 12

☆ 103

🔖 1

💬 39



Q



目录



53



12



103



1



39